LEGPHEL PMS

DEV-SPEC-001

Part 1 — Document Identity, Derivation, and Enforcement

§1.1 through §1.9

Status: DRAFT — review flags resolved; awaiting Architect lock confirmation

Canon version: v2.5

Authority: MOM-ARCH-2026-013

Gate: Gate 1 — Document Identity

Date: 07 April 2026

Prepared by: Claude (AI Architectural Partner)

This document is the architectural translation layer between Canon v2.4 and the Stage Implementation Guidelines. It expresses what the Canon says as implementation obligations — concrete, enumerated, and stated with enough precision that an LLM can generate correct code without guessing at business meaning.

LEGPHEL PMS — DEV-SPEC-001

Part 1 — Document Identity, Derivation, and Enforcement

§1.1 through §1.9

Document: DEV-SPEC-001-Part1.md

Status: DRAFT — review flags resolved; awaiting Architect lock confirmation

Canon version: v2.5

Authority: MOM-ARCH-2026-013

Gate: Gate 1 — Document Identity

Date: 07 April 2026

Prepared by: Claude (AI Architectural Partner)

Document Control

Field	Detail
Gate	1 — Document Identity
Sections covered	§1.1 through §1.9
Declared Canon sources	Block 1 (§§1–16), Block 2 §§17–21, Block 4 §34, Canon v2.4 changeset (§80 Group column, §70 Resolution Bundle, §71 mutation rules)
Companion documents	ACTOR-AUTHORITY-MATRIX-LOCKED.md (LOCKED — MOM-ARCH-2026-012)
Status	DRAFT — nothing is locked until Architect confirms
Previous gate	None — Gate 1 is the first writing gate
Depends on	Actor-Authority Matrix LOCKED ✓, Canon v2.4 confirmed ✓

---

§1.1 — Purpose and Position in Build Chain

1.1.1 What This Document Is

DEV-SPEC-001 is the architectural translation layer between Canon v2.4 and the Stage Implementation Guidelines. Its sole function is to express what the Canon says as implementation obligations — concrete, enumerated, and stated with enough precision that an LLM can generate correct code against Stage Implementation Guidelines without guessing at business meaning.

LLMs generate code against Stage Implementation Guidelines. Stage Implementation Guidelines derive from DEV-SPEC-001. DEV-SPEC-001 derives from Canon v2.4 and the Actor-Authority Matrix. The Canon and the Actor-Authority Matrix are the sources of business truth. This chain is the governing build chain for all implementation work on the LEGPHEL PMS.

The build chain is:

```
Canon v2.4
  → Actor-Authority Matrix (companion document, LOCKED)
    → DEV-SPEC-001 (this document)
      → Stage Implementation Guidelines
        → Code
```

No layer in this chain may redefine the truth of a higher layer. Code does not define business logic. Stage Implementation Guidelines do not redefine what a stage may do. DEV-SPEC-001 does not narrow or expand what the Canon says. Where any layer contradicts a higher layer, the higher layer governs and the lower layer is wrong.

1.1.2 The Zero-Gap Standard

DEV-SPEC-001 operates under the zero-gap standard. Every pattern the implementation must follow must be shown in this spec. Every pattern the implementation must not follow must be stated in this spec. Any gap in this spec is a gap that will be filled by an LLM with generic patterns that are not Canon-compliant. Generic patterns are the failure mode — they are architecturally coherent but operationally wrong.

A gap is not the same as a deliberate deferral. Deliberate deferrals are surfaced as Appendix A clarification requests with a Category classification. Gaps are absences that appear to be complete coverage but are not. The zero-gap standard requires that gaps be eliminated, not hidden.

1.1.3 Completion and Hardening, Not Greenfield

This system is a refactoring of a working codebase. DEV-SPEC-001 is a completion and hardening specification. It does not describe an ideal system. It defines the governed architecture that the working system must be made to embody.

The implications are concrete:

- Existing code that complies with DEV-SPEC-001 is preserved.
- Existing code that contradicts DEV-SPEC-001 is corrected.
- Existing code that addresses a concern DEV-SPEC-001 does not cover is examined against the Canon before being retained or removed.
- No feature is added merely because the Canon permits it. Each feature is added because the Canon requires it.

1.1.4 System Character: Operational Intelligence, Not Passive Recording

This PMS is not a passive recording system. It carries operational intelligence. It guides the operator, prevents invalid action, preserves history, and makes the correct next action obvious. It teaches correct hotel behaviour, enforces governed procedures, and structures operational motion.

The implications for implementation:

- Every controlled operation must be enforced at the service layer, not merely displayed in the UI. A UI element being disabled is not enforcement. The service must reject the invalid request regardless of what the UI shows.
- Every governed state change emits a trace event in the same database transaction. If the transaction rolls back, the trace event rolls back with it.
- Every point of ambiguity in operational flow is resolved by the system with a typed, actionable response — not an unhandled exception, not a generic error, not silence.
- The system surfaces the current state of every active item and every pending obligation to the operator who opens it. The operator does not need to ask what happened while they were away.

1.1.5 IP Boundary Rule

This spec carries rules. It does not carry their provenance. No architectural principle numbers, no references to the foundational architecture canon, and no source document attribution appear in this spec. Rules are stated as implementation obligations. Their derivation traces to Canon sections and MOM decisions per §1.2. Principle numbers are provenance annotations in the Canon. They are not implementation obligations in this spec.

---

§1.2 — Derivation Rules

1.2.1 Declared Source Documents

Every obligation in DEV-SPEC-001 traces to one of the following declared source documents:

Source	Description
Canon v2.4	Operational Workflow & Control Canon, all 11 blocks, including v2.3 and v2.4 changesets. The authoritative source of business truth for the LEGPHEL PMS.
MOMs 003–012	Minutes of Meeting records covering architectural deliberation from MOM-ARCH-2026-003 through MOM-ARCH-2026-012. Decisions locked in MOMs are binding unless formally revised through a new MOM.
Actor-Authority Matrix (LOCKED)	ACTOR-AUTHORITY-MATRIX-LOCKED.md, reviewed and locked in MOM-ARCH-2026-012. Companion document to Canon v2.4. Formally part of the build chain.

Nothing in DEV-SPEC-001 is invented by the gate writer. Nothing is inferred from operational intuition. Nothing is filled from general software engineering convention without a Canon or MOM source. Where a rule cannot be derived from the declared sources without guessing, it is surfaced as an Appendix A clarification request per §1.3.

1.2.2 Truth Model Hierarchy

The LEGPHEL PMS operates on a deliberate truth model that every implementation decision must respect:

Documents define meaning. The Canon and MOMs define operational meaning. Code does not define business logic. Developer interpretation is not a source of business truth.

Policies define rules. Policy classes (§72) define the specific rules governing operational decisions. Policy rules bind code. Code does not relax policy rules.

Configuration enables controlled variation. Configuration surfaces in the Admin Console allow the hotel to tune behaviour within policy bounds. Configuration does not override policy. Configuration does not rewrite Canon truths.

Code executes intent. Code realises what the spec says. Code does not add, remove, or reinterpret business meaning.

Data records history. Records are the permanent, append-only evidence of what happened. Data does not define what should happen next.

This order is structural, not stylistic. Lower layers cannot redefine the truth of higher layers. A configuration threshold is not authority to bypass a policy. A code path is not a business rule. A current database state is not a licence to skip a stage gate.

1.2.3 Derivation Obligation

Every obligation stated in this spec is derivable from its declared Canon source without guessing. Where this spec says "the system must", a specific Canon section, MOM decision, or Actor-Authority Matrix row supports that statement. Appendix A accumulates the full cross-reference table.

Gate writers are bound by this rule in both directions: they may not invent obligations not in the Canon, and they may not omit obligations that are in the Canon. The derivation is mechanical: read the declared source, extract the obligation, state it in implementation terms.

1.2.4 IP Boundary Rule (Stated)

The Canon carries citations from its own source documents as provenance annotations. Those citations are not implementation obligations. This spec carries the rules, not their provenance. No source document attribution from above the Canon in the documentation stack appears in this spec or in any downstream gate output.

1.2.5 Category 1 Policy Gaps

Where a business rule cannot be derived from the declared Canon sources without guessing, the gate writer surfaces a Category 1 clarification request in Appendix A. Category 1 items require Architect deliberation before the relevant gate proceeds. The gate writer does not choose. The gate writer does not infer. The gate writer flags and waits.

---

§1.3 — Spec Ambiguity Protocol

1.3.1 Appendix A Protocol

When a gate writer encounters a business rule they cannot derive from the declared Canon sources without guessing, the correct action is to surface a clarification request in Appendix A. The gate writer does not choose the more convenient interpretation. The gate writer does not infer from adjacent rules. The gate writer does not mark the item as a known limitation and move on.

The clarification request must state: the section it blocks, the question that cannot be answered from the declared sources, the candidate interpretations (if more than one exists), and the Architect's decision once made.

No section may be considered complete if it contains a business rule that was inferred rather than derived.

1.3.2 Category 1 — Structural Gaps

Category 1 clarification requests are structural gaps — business rules that are absent from the Canon or ambiguous in the Canon such that the gate writer cannot produce a compliant implementation without inventing policy content. Category 1 items block the affected gate section from being considered final. The Architect must deliberate and lock a decision before the blocked section is written.

Category 1 items accumulate in Appendix A and feed into Appendix B (Spec Ambiguity Register). Open items from prior sessions already registered in Appendix B include: B4-001 (concurrent editing technical mechanism), B9-001 (amendment routing algorithm — Path 1/2/3 classification basis), B9-003 (S9-equivalent processing for cancelled entries with pending refunds), B2-005 and B9-005 (AWAITING_WRITTEN_CONFIRMATION state machine completeness), B3-001 (QUOTED displacement threshold), and B11-001 (Entry to CLOSED terminal condition commission-due guard).

1.3.3 Category 2 — Configurable Thresholds

Category 2 items are not Canon gaps. They are configurable parameters — numeric thresholds, authority bands, timer durations, and similar values — that are managed through the Admin Console and do not require Architect

deliberation to assign. Category 2 items carry placeholder default values and are accumulated in Appendix C (Seed Data Specification).

Examples of Category 2 items: specific discount threshold percentages per authority level, credit ceiling authority thresholds per client tier, rate override margin values per rate plan, QUOTED displacement threshold.

1.3.4 Blocking Evidence vs Quality Signals

This distinction governs stage exit behaviour and must be stated correctly in every state machine section of this spec. It must not be resolved by the gate writer choosing the more convenient classification.

Blocking evidence is evidence whose absence prevents stage exit. The system must not permit the stage transition to proceed when blocking evidence is unsatisfied. A blocking evidence gap produces a `StageGateBlockedError`, not a warning. There is no option to proceed anyway without a formal escalation path.

Quality signals are indicators that the system surfaces and recommends but that do not prevent stage exit when absent. The system may warn. The system may require acknowledgement. The system must not block.

Downgrading blocking evidence to a quality signal for convenience is forbidden. The Canon designates the classification for each exit condition at each stage. Where the Canon does not clearly classify a condition, the gate writer surfaces it as a Category 1 clarification request. The gate writer does not choose a classification by inference.

1.3.5 Hard Boundary Language

This spec must use hard boundary language wherever a hard boundary exists. "Must" means the system enforces it. "Is forbidden" means the system rejects it. "Requires" means the condition is a precondition. Soft language — "should", "is recommended", "generally", "where possible" — is not used for controlled operational behaviour. Soft language may be used only for informational or guidance content where no enforcement point exists.

---

§1.4 — Forbidden Implementation Patterns

This section enumerates implementation patterns that are structurally forbidden. These are not guidelines. Each pattern listed here, if present in the codebase, is an architectural defect that must be corrected. The patterns are organised by system concern. The Canon source is cited for each.

1.4.1 Schema-Level Forbidden Patterns

Silent mutation of any committed field. Once a record is committed (see immutability rules per §71 and stage-specific seal events), no field on that record may be changed in place. Corrections are additive layers — new records, new versions, amendment layers — that preserve the original and explain the modification. The original record and its original values must remain in the permanent record. This is the governing forbidden act for all amendment paths (§12, §51 M.13).

In-place editing of commitment snapshot after S4 exit. The commitment snapshot created at S4 confirmation contains the frozen commercial terms of the engagement — rate, inclusions, conditions. After S4 exit, the commitment snapshot is sealed. Any code path that modifies a field on the commitment snapshot after it is sealed is the most dangerous schema-level violation in the system. Re-entry is the only path to change commercial terms post-S4 (§45.18).

Deletion of any operational record. Operational records are never deleted. CANCELLED is a state transition, not a deletion. CLOSED is a state transition, not a deletion. An entry that reaches a terminal state retains its complete history in the database indefinitely. Any DELETE statement against an operational record table is forbidden to all

actors including system workers (§12, §52 M.13).

Backdating post-closure entries to original stay dates. Post-closure additive records — post-stay charges, credit notes, commercial adjustment entries — carry their actual transaction date. They are never given a date within the original stay period. Night audit records are authoritative for their date and are never modified post-seal (§7, §29.5, §50.18).

Modifying sealed night audit records. Once a night audit record is sealed for a date, it is immutable. No catch-up charge posting, no correction, and no audit adjustment may modify the sealed record. Post-seal adjustments are new records with their own dates (§62 M.13).

Modifying sealed folio lines. Folio lines are immutable from creation. Post-settlement adjustments are new additive records. No UPDATE statement against a folio line after it is posted is permitted (§49.18).

1.4.2 State Machine Forbidden Patterns

Bypassing a stage gate. Stage gates are hard blocks, not warnings. If a gate condition is unsatisfied, the state machine must reject the transition. No service method, no API endpoint, and no internal call path may circumvent a stage gate check. Hard blocks are not advisory. Escalation is the only path when the normal path is blocked — there is no "force proceed" path without a formal escalation authority and a recorded reason (§38, Actor-Authority Matrix Part 1).

Soft completion — downgrading blocking evidence to a quality signal. Blocking evidence for stage exit is defined by the Canon for each stage. An implementation that reclassifies a Canon-designated blocking condition as a warning for implementation convenience has weakened the stage gate. This produces a system that can be soft-completed — driven through stages without satisfying their exit requirements. Any code path that produces a warning where the Canon requires a block is a violation (§38.4).

Missing state machine state on auto-fulfilled transitions. Modes may auto-fulfil stage transitions, but the state machine must still record each stage passage. Auto-fulfilment is not stage skipping. The audit trail must show each stage the entry passed through, including auto-fulfilled stages, with the system actor as the attributed actor for each (§21).

Auto-confirming a no-show without FOM determination. The Timer Engine may fire a no-show flag after the configured contact-attempt window expires, but no system worker or automated path may formally classify an entry as a no-show without a FOM determination event. No-show is a human decision executed through a governed workflow (§46.18, §54 M.13, Actor-Authority Matrix No-Show Mechanism).

1.4.3 Service-Level Forbidden Patterns

External systems writing to PMS tables. The PMS operates on an export model, not an integration model. External systems receive data from the PMS through defined export paths. No external system has write access to any PMS table. OTA bookings arrive as inbound data that staff manually enter — no OTA system writes to the PMS directly (§33.3).

Admin Console code creating operational records. The Admin Console is a configuration authority surface. It configures behaviour through managed configuration tables. It does not create inquiries, entries, folios, charges, reservations, or any operational record. Any admin service that calls an operational record creation method is an architectural break (§7, §17, §18).

Silent expiry of any governed condition. Every expiry of a governed condition — hold timer expiry, quotation validity expiry, parking follow-up expiry, processing lock expiry, advance payment window expiry — is a governed event that produces an audit record. A worker that transitions a record from an active state to an expired state without emitting an audit event is forbidden. Expiry is always a governed event with a record (§32.4).

Decisions outside stage context. Any code path that creates or modifies an operational record without an active stage context is an architectural violation. Post-closure access is the governed exception — it is anchored to the entry and its S9 closure record, not floating outside the stage model. The night audit is a cross-stage system actor whose effects are stage-specific (§7).

Silent inventory release. Inventory state transitions from a held or confirmed state to FREE must be the system consequence of a governed cancellation or closure event with an audit record. No code path may release inventory silently — without a triggering event, a record of that event, and an attributed actor (§52 M.13, Actor-Authority Matrix Cancellation Mechanism).

Operational staff self-serving coordinator requests. Coordinator amendments to work orders are always mediated by hotel staff. No external-actor request (coordinator, agent, guest) may directly trigger a write path in the PMS. All external-actor requests are initiated by operational staff as system actors (§65 M.4, §56 M.5).

1.4.4 Engine-Level Forbidden Patterns

Bypassing the PricingPipelineEngine for rate computation. All rate computation passes through the PricingPipelineEngine. No service, controller, or worker computes a rate by any other means. Rate override actions (FOM margin adjustment, GM override) are inputs to the PricingPipelineEngine, not bypasses around it (§43.18, §51 M.13).

Making rate plan priority order configurable. The rate plan priority order — individual-level agreements > promotional > tier-level > channel-level > rack — is hardcoded PricingPipelineEngine behaviour. It is not a configurable parameter. No Admin Console surface exposes this order for hotel adjustment (§30.2).

Conflating engine hardcoded behaviour with configurable parameters in documentation or code. Engine hardcoded behaviour is the logic the engine always executes regardless of input configuration. Configurable parameters are the inputs the engine reads. Documentation and code must clearly separate these categories. A comment, method name, or configuration key that implies an engine behaviour is configurable when it is not is a documentation violation that will produce incorrect downstream code.

1.4.5 Worker-Level Forbidden Patterns

Posting charges after night audit has sealed for that date. Workers that post room charges, service charges, or any financial line must check whether the night audit for the relevant date has been sealed before posting. Posting to a sealed date is forbidden. Charges for a sealed date are AUDIT_EXCEPTION items requiring FOM approval for catch-up posting as new records with their actual posting date (§48.18, §62 M.13).

Auto-closing outstanding payment balances. Outstanding folio balances may not be automatically closed, written off, or marked as settled by any worker. Write-off requires GM authority and a mandatory recorded reason. Automatic settlement of unpaid balances is forbidden (§50.18).

Non-idempotent worker execution. All workers must be idempotent. A worker that runs twice on the same job must produce the same result as a worker that ran once. Workers must check whether their effect has already been applied before applying it. Double-execution producing duplicate records, duplicate charges, or duplicate events is a worker-level architectural defect.

1.4.6 Admin Console Forbidden Patterns

Admin Console processing stage workflows. The Admin Console does not process check-ins, generate confirmation vouchers, post financial charges, or execute any operational stage action. Admin Console code paths are restricted to configuration table reads and writes. Any Admin Console route that calls an operational service is a boundary violation (§18).

Admin Console namespace importing operational services. Admin code must not import operational service modules. Operational code may read from configuration tables but must not call admin service methods. The import direction is one-way: admin reads and writes configuration; operations reads configuration (§18).

Operational code calling Admin Console services. The separation is bidirectional. Operational services read from configuration tables through standard database access. They do not call Admin Console service methods. Importing admin services into operational services is a dependency violation (§18).

1.4.7 Controller-Level Forbidden Patterns

Business logic in controllers. Controllers are thin HTTP adapters. They parse inbound requests, invoke the correct service method, and format the outbound response. Business rules, policy checks, state machine logic, financial calculations, and authority decisions must not appear in controller code. A controller that contains a conditional implementing a business rule is an architectural violation — the rule belongs in the service or policy layer where it is testable in isolation (§13.8).

Bypassing auth middleware. Every API endpoint passes through authentication and authorisation middleware before the controller handler executes. No endpoint may be exposed without this middleware chain. An endpoint that directly handles a request without validating the authenticated session and the actor's authority level is a security and audit violation (§34.4).

Bypassing validation middleware. Input validation middleware executes before the controller handler. No controller handler may receive unvalidated input. Validation of request shape, required fields, and type constraints happens in middleware — not as ad-hoc checks inside the controller handler (§13.8).

Non-standard response envelope. All API responses follow the standard response envelope (success flag, data payload or error payload, requestId for audit correlation). No controller may return a raw object, a naked array, or an error without the typed error envelope defined in §1.5.4. Inconsistent response shapes break the mechanical derivation chain from spec to consumer code.

Controller managing transactions. Database transaction management belongs in the service layer, not the controller layer. A controller that opens, commits, or rolls back a database transaction directly is violating layer separation. The service layer ensures that trace events and state changes are committed atomically.

1.4.8 AI Agent Forbidden Patterns

AI agent approving its own drafts. The AI agent produces drafts. Human approval is a required separate event. No code path allows the AI agent to send a communication without a human approval event being recorded as a distinct prior action attributed to a human actor. The approval cannot be implicit, inferred, or system-generated (§70B M.5, Actor-Authority Matrix AI Communication Agent).

AI agent sending communication without a human approval event. This is absolute. No trust level configuration — including FULL_AUTO — overrides the requirement for human approval before outbound communication is sent. FULL_AUTO trust level applies to internal system actions only. It is expressly forbidden for outbound communication (§70B M.5, M.6).

AI agent processing voice note content. Voice notes received by the system are flagged as VOICE_NOTE type on the Communication Record. The AI agent is blocked at the routing layer from receiving voice note content for processing. This is a routing-layer block, not a runtime check. The AI agent must never receive a voice note as input for intent classification or draft generation (§70C M.5).

AI agent executing governed actions autonomously. The AI agent may propose system actions — record creation, field updates, stage transitions, work order items. Proposals require human approval at the appropriate authority level for the specific action. The AI agent executing a governed action autonomously — regardless of action category or

trust level — is forbidden (§70B M.5).

1.4.9 OTA Integration Forbidden Patterns

Auto-confirming OTA bookings without human verification. OTA bookings arrive as inbound data (currently via confirmation emails entered manually). No code path, worker, or integration service automatically confirms an OTA booking without a human verification step being recorded (§64 M.13).

Classifying OTA_CONFLICT overbooking as DELIBERATE. When an overbooking is detected and the triggering entry carries the OTA_SOURCE flag, the trigger type is immutably set to OTA_CONFLICT. No actor, no service, and no worker may override this classification to DELIBERATE. The OTA_CONFLICT trigger type is set by the system at detection and is immutable from creation (§64 M.13, Actor-Authority Matrix S4).

1.4.10 Authentication and Attribution Forbidden Patterns

Shared credentials. Two staff members sharing one login account is a system design failure. The authentication model makes individual authentication fast enough (PIN-based fast switching) that sharing is unnecessary. Any authentication design that permits credential sharing makes attribution impossible and is architecturally forbidden regardless of convenience argument (§34.4).

Retrospective attribution change. Once an action is recorded with an attributed actor, the attribution is permanent. No code path, no admin function, and no data migration may change the actor on a recorded action. Corrections to business decisions are new actions by new actors — the original action and its attribution remain in the permanent record (§34.4).

Implicit "on behalf of" attribution. There is no "on behalf of" mechanism. If Staff A takes an action while authenticated as Staff A, the record permanently shows Staff A. Instructions from a supervisor are not an attribution source. If supervisory direction is captured at all, it is a separate contextual note — not the attributed actor on the primary record (§34.4).

1.4.11 Stage-Specific Forbidden Patterns

Creating any inventory hold at S1. No hold — speculative or committed — may be placed at Stage 1. Inventory is not committed at S1. Rate indication at S1 is advisory only and must not be presented as final. The forbidden acts at S1 are: any inventory hold, any folio creation, any payment collection, any quotation issuance, and any final rate communication (§42.18).

Issuing confirmation voucher at S3. S3 is Reservation Setup. Confirmation is S4's responsibility. No confirmation voucher, confirmation language, or confirmation communication may be issued at S3. The provisional folio created at S3 accepts advance payments and proforma invoices — it does not accept live charges (§44.18).

Room-specific assignment at S2, S3, or S4. Room assignment happens at S5 (Pre-Arrival). No specific room number may be promised or assigned to a guest at S2, S3, or S4. Availability searches at S1 produce type-level availability, not room-specific commitments (§43.18, §44.6, §45.6).

Notifying a guest of changed commercial terms without a system-generated amended document. Any amendment that changes the commercial terms communicated to a guest requires a system-generated amended document — a new quotation version, an amended confirmation, or a formal amendment communication. Verbal notification or informal message without a system-generated document is forbidden (§51 M.13).

Posting charges after night audit has sealed for the current date. This appears in both worker-level patterns (§1.4.5) and here as a stage-specific pattern because it applies to any S7 charge posting path. Stage 7 charge operations must validate the audit seal status for the target date before posting (§48.18).

Charging a guest for damage in a room that carried a DEFICIENT flag without FOM review. If a room was assigned with a DEFICIENT flag in place, and a DEFICIENT condition is found at room inspection, FOM must review the condition before the system permits a damage charge against the guest. Charging a guest for a pre-existing condition is forbidden without FOM determination (§49.18).

Allowing engagement departure with a BLOCKED dispute without a DisputeGateOverrideRecord. If a dispute is in OPEN or IN_PROGRESS state at S8, the stage gate blocks departure. The only path through this block is a DisputeGateOverrideRecord created by GM. No other actor has override authority. No dispute may be skipped by deletion, status manipulation, or informal bypass (§53 M.8, §49.18).

Closing engagement with an active unresolved dispute. A dispute in OPEN or IN_PROGRESS state blocks S9 engagement closure. The dispute must reach a terminal state (RESOLVED or CLOSED) before engagement closure can proceed. DISPUTE_EXHAUSTED is not a valid dispute state — it does not exist in the system (§53 M.12).

---

§1.5 — Error Taxonomy

1.5.1 Error Design Principle

Every error produced by this system is typed, specific, and actionable. Generic exception messages — "Something went wrong", "Unexpected error", "Internal server error" without a typed payload — are not acceptable error outputs for any controlled operational condition. Every error type carries a structured context payload that identifies the specific entity, condition, and resolution path.

The existing error class hierarchy is preserved and extended.

1.5.2 Existing Error Classes (Preserved)

These error classes exist in the current codebase and are retained:

Class	Purpose
`AppError`	Base class for all application-level errors. Carries `code`, `message`, `statusCode`, and `context` payload. All classes below extend this.
`ValidationError`	Input validation failure. Carries field-level detail: `field`, `value`, `constraint`, `message`.
`NotFoundError`	Entity not found. Carries `entityType`, `entityId`.
`PolicyViolationError`	A business rule was violated. Carries `policyName`, `violatedCondition`, `actorLevel`, `requiredLevel`.
`AuthorizationError`	Actor does not have authority for the requested action. Carries `action`, `actorLevel`, `requiredLevel`, `escalationPath`.
`StateTransitionError`	A state machine transition was attempted in an invalid state. Carries `entityType`, `entityId`, `currentState`, `attemptedTransition`, `requiredConditions`.

1.5.3 New Error Types Required by Canon

These error types are not present in the existing codebase and must be introduced:

`MissingConfigurationError`

Thrown when a governed operational decision cannot be executed because required configuration is absent or invalid. This error surfaces the specific configuration gap — it does not produce a generic "configuration error". Configuration failure must always identify exactly what is missing (§19, §20).

Context payload:

```
{
  configurationKey: string,       // exact configuration surface that is missing or invalid
  configuredBy: string,           // role responsible for providing this configuration (e.g., "GM_ADMIN_CONSOLE")
  affectedOperation: string,      // operation that cannot proceed without it
  stage: string | null,           // stage this operation belongs to, if applicable
  resolutionPath: string          // human-readable instruction for resolution
}
```

`PolicyGateBlockedError`

Thrown when a policy check blocks an operation that has not been escalated to sufficient authority. Distinct from `AuthorizationError` — this error carries the specific policy, the specific condition that is unsatisfied, and the escalation path (§72).

Context payload:

```
{
  policyName: string,             // canonical policy class name
  policySection: string,          // Canon section reference for this policy
  blockingCondition: string,      // precise statement of what is unsatisfied
  currentActorLevel: string,      // L0–L4 or EXT
  requiredActorLevel: string,     // minimum authority level for this action
  escalationPath: string,         // what the actor must do next
  entryId: string | null          // anchor entry if applicable
}
```

`StageGateBlockedError`

Thrown when a stage exit is attempted and one or more blocking evidence conditions are unsatisfied. Carries sufficient detail for the system to surface the specific unmet conditions and, where applicable, whether an override path exists (§76A, Actor-Authority Matrix).

Context payload:

```
{
  stage: string,                  // e.g., "S3"
  attemptedTransition: string,    // canonical transition name per §76A
  unsatisfiedConditions: Array<{
    condition: string,            // description of the unmet condition
    canonSource: string,          // Canon section reference
    overrideAvailable: boolean    // whether BLOCKED_WITH_OVERRIDE_AVAILABLE applies
  }>,
  guardOutcome: "BLOCKED" | "BLOCKED_WITH_OVERRIDE_AVAILABLE",
  overrideRequires: string | null // e.g., "GM authority — DisputeGateOverrideRecord required"
}
```

`ConcurrentEditingError`

Thrown when two sessions attempt to edit the same entry simultaneously and the system detects the conflict. Surfaces the concurrent access condition to both users. The technical mechanism for detecting concurrency (optimistic locking, presence indicator, micro-hold) is a §9.2 gate concern. The error type is defined here as a requirement. This error must never result in silent data loss or a composite state from two concurrent edits (§34.4, Appendix B item B4-001).

Context payload:

```
{
  entityType: string,             // e.g., "Entry"
  entityId: string,
  currentEditorSession: string,   // session identifier of the currently editing session
  conflictDetectedAt: string,     // ISO timestamp
  resolution: string              // instruction for the actor who encounters the conflict
}
```

`OverbookingDetectedError`

Thrown when the overbooking detection framework identifies an overbooking condition at S4 confirmation. Carries the trigger type — either DELIBERATE (hotel-initiated overbooking) or OTA_CONFLICT (OTA synchronisation gap). The trigger type is immutable once set and must never be overridden by a subsequent operation (§45.5, §28.1, §64).

Context payload:

```
{
  triggerType: "DELIBERATE" | "OTA_CONFLICT",
  entryId: string,
  conflictingEntries: string[],   // entry IDs that conflict with this booking
  requiresGmApproval: true,       // GM approval is mandatory for all overbooking paths
  mitigation: string | null       // mitigation path if already in progress
}
```

1.5.4 Error Envelope Standard

All API error responses follow the standard error envelope. No raw exception objects or unstructured strings are returned to callers:

```
{
  success: false,
  error: {
    type: string,         // error class name
    code: string,         // machine-readable error code
    message: string,      // human-readable message
    context: object       // typed context payload per error class above
  },
  requestId: string       // correlation identifier for audit trail linkage
}
```

---

§1.6 — Controlled Vocabulary and Canonical Terms Registry

This section registers the canonical names, enum values, and term definitions that all implementation code must use. Code that uses non-canonical terms for these values introduces a naming inconsistency that breaks the mechanical derivation chain from spec to implementation to audit.

> ***Naming convention note:*** *MOM-ARCH-2026-007 §5.1 records a deliberated Inquiry→Engagement rename. The Canon consistently uses "Inquiry" throughout all 11 blocks. Implementation code should confirm the settled term at gate before finalising model and field names. "Inquiry" is used throughout this spec; the gate writer surfaces this as a confirmation request for the Architect.*

1.6.1 Stage Identifiers

Identifier	Canonical Name	Description
S1	Inquiry & Configuration	Initial engagement; availability search; guest and use type capture

Identifier	Canonical Name	Description
S2	Negotiation & Quotation	Rate negotiation; quotation creation and versioning; hold placement
S3	Reservation Setup	Committed hold; provisional folio; advance payment; work order initiation
S4	Confirmation & Ownership	Commercial terms frozen; confirmation voucher issued; inventory locked
S5	Pre-Arrival / Pre-Event	Room assignment; pre-arrival communication; no-show detection
S6	Check-In & Stay/Event Initiation	Identity verification; folio activation; handoffs H2/H3
S7	In-Stay / In-Event	Daily charges; night audit; room change; amendments
S8	Checkout & Settlement	Folio presentation; settlement; room inspection; H5 handoff
S9	Post-Stay & Closure	Invoice dispatch; payment matching; post-stay corrections; engagement closure

1.6.2 Inventory Claim States

State	Meaning
`FREE`	No claim on this inventory unit. Available for assignment.
`QUOTED`	Quoted in an active S1/S2 session. Non-binding.
`SPECULATIVELY_HELD`	Speculative hold placed at S2 with FOM/GM authority. Timer-governed.
`COMMITTED_HELD`	Committed hold placed at S3. Commercially justified. Timer-governed with different expiry rules.
`CONFIRMED`	Reservation confirmed at S4. Inventory is locked to this entry.
`OCCUPIED`	Guest checked in. S6 through S8.
`DEPARTED_DIRTY`	Guest checked out. Awaiting housekeeping.
`DEPARTED_CLEAN`	Housekeeping completed. Room returned to available status pending inspection.
`BLOCKED`	Administratively blocked from booking. Requires governed unblock event.
`UNDER_MAINTENANCE`	Room undergoing maintenance. Must carry governed deadline for return to service.

1.6.3 Engagement Use Types

The use type is an attribute on the Entry record, selected at S1 and carried throughout the lifecycle. It does not create a separate lifecycle. All use types pass through the same S1–S9 state machine (§27).

Value	Description
`ACCOMMODATION`	Standard room or suite booking

Value	Description
`CONFERENCE`	Space-centric engagement with hall, seating, and F&B components
`APARTMENT`	Extended-duration occupancy with periodic billing and lease terms
`CATERING`	Off-site F&B service delivery; no room occupancy
`GROUP`	Multiple-guest booking with coordinator, rooming list, and billing mode governance

1.6.4 Actor Levels and Role Identifiers

Level	Identifier	Canonical Role Name
`L0`	`SYSTEM`	Automated system actor — Timer Engine, night audit workers, expiry workers
`L1`	`FRONT_DESK` / `CUSTODIAN`	Operational staff — front desk, reservations
`L2`	`FOM`	Front Office Manager
`L3`	`GM`	General Manager
`L4`	`ADMIN`	Configuration authority — Admin Console access only
`EXT`	`EXTERNAL`	External actor (coordinator, agent, guest) — no direct system access

1.6.5 Stage Gate Guard Outcomes

These are the only valid outcomes of a stage gate evaluation (§76A):

Value	Meaning
`CLEAR`	All guard conditions satisfied. Transition may proceed.
`BLOCKED`	One or more guard conditions unsatisfied. No override path exists.
`BLOCKED_WITH_OVERRIDE_AVAILABLE`	Guard condition unsatisfied but a formal override path exists (requires GM authority and `DisputeGateOverrideRecord`).

1.6.6 Trust Level Values (AI Agent)

These are the only valid trust level configurations for AI agent action categories (§70B):

Value	Meaning
`ALWAYS_REQUIRE_APPROVAL`	Default. All AI-drafted outputs require explicit human approval before execution or sending.
`AUTO_APPROVE_HIGH_CONFIDENCE`	System may auto-approve when AI confidence score exceeds configured threshold. Human review still available.
`FULL_AUTO`	Permitted for internal system actions only when GM-configured. Expressly forbidden for all outbound communications.

1.6.7 OTA-Specific Identifiers

Identifier	Type	Description
`OTA_SOURCE`	Flag on Entry	Set by system at S1 when booking source is an OTA channel. Immutable once set. No actor may override or unset.
`OTA_CONFLICT`	Overbooking trigger type	Set when an overbooking is detected and the triggering entry carries `OTA_SOURCE`. Immutable once set.
`DELIBERATE`	Overbooking trigger type	Set when an overbooking is detected and no `OTA_SOURCE` flag is present (hotel-initiated commercial overbooking).

1.6.8 Folio States and Closure Types

State / Type	Context	Meaning
`PROVISIONAL`	Folio state at S3	Accepts proforma invoices and advance payments. Does not accept live charges.
`LIVE`	Folio state at S6+	Activated at check-in. Accepts all stay charges, credits, and adjustments.
`SETTLED`	Folio closure type	All charges settled. Standard closure path.
`NO_SHOW_CLOSED`	Folio closure type	Guest did not arrive. Folio closed with penalty and advance payment recorded. Immutable from closure.
`OUTSTANDING`	Folio closure type	Balance remains unpaid at S9 closure. Open for post-stay follow-up.
`CANCELLED`	Folio state	Engagement cancelled before check-in. Folio carries advance payment history.

1.6.9 Invoice States and Types

Value	Context	Meaning
`CREATED`	Invoice state	Invoice generated but not dispatched
`DISPATCHED`	Invoice state	Invoice sent to guest/agent/corporate
`PAYMENT_TRACKED`	Invoice state	Payment received against this invoice; matching in progress
`RECONCILED`	Invoice state	Payment fully matched; invoice closed
`PROFORMA`	Invoice type	Pre-commitment invoice issued at S3
`FINAL`	Invoice type	Settlement invoice issued at S8/S9
`POST_STAY`	Invoice type	Post-closure invoice for delayed billing (corporate/government)

1.6.10 Handoff States and Types

State	Meaning
`CREATED`	Handoff created by sending party with required content

State	Meaning
ASSIGNED	Directed to specific receiving actor or role
ACCEPTED	Receiver has acknowledged receipt and taken accountability
FULFILLED	Receiver has completed the obligated work with required evidence
CLOSED	Verified complete. No residual obligation.
REJECTED	Receiver cannot accept due to blocking conditions
ESCALATED	Stalled past SLA; routed to management by Timer Engine

Type	Transfer
H1	Reservations → Front Desk (S4→S5 transition)
H2	Front Desk → Housekeeping (S6 check-in)
H3	Front Desk → F&B (S6 check-in)
H4	Pre-checkout coordination — Housekeeping/F&B → Front Desk (approaching S8)
H5	Front Desk → Accounts/Reservations (S8→S9 transition)

1.6.11 Dispute States and Failure Categories

State	Meaning
OPEN	Dispute filed; FOM owns and manages
IN_PROGRESS	Active resolution in progress
RESOLVED	Guest accepted resolution
CLOSED	GM formal closure with mandatory recorded reason
REOPENED	Reopened by FOM or GM with mandatory reason

DISPUTE_EXHAUSTED *is explicitly not a valid dispute state. It must not appear in the implementation (§53 M.12).*

Failure Category	Code
Room condition failure	ROOM_CONDITION
Service delivery failure	SERVICE_DELIVERY
Communication failure	COMMUNICATION
Billing dispute	BILLING
Commercial terms dispute	COMMERCIAL_TERMS
Operational process failure	OPERATIONAL_PROCESS

1.6.12 AI Draft Record and Human Decision Record Values

Context	Value	Meaning
AI Draft Record status	PENDING_REVIEW	Draft awaiting human review
AI Draft Record status	APPROVED	Approved without changes

Context	Value	Meaning
AI Draft Record status	`EDITED_AND_APPROVED`	Human edited draft before approving
AI Draft Record status	`REJECTED`	Rejected; reason mandatory
HDR decision type	`APPROVE`	Approved as-is
HDR decision type	`EDIT_AND_APPROVE`	Edited and approved; final content recorded
HDR decision type	`REJECT`	Rejected; reason mandatory

1.6.13 Communication Channel and Status Values

Context	Value	Meaning
Channel	`EMAIL`	Email communication via Amazon SES / IMAP
Channel	`WHATSAPP`	WhatsApp Business API via BSP
Channel	`PHONE`	Phone call (Android call log integration)
Channel	`OTA`	OTA platform message
Message type	`STANDARD`	Standard text message
Message type	`VOICE_NOTE`	Voice note — routed to human-only review path (§70C)
Send status	`DRAFT`	Created, not queued
Send status	`QUEUED`	In outbound queue
Send status	`SENT`	Dispatched from queue
Send status	`DELIVERED`	Delivery confirmed
Send status	`BOUNCED`	Delivery failed with bounce event
Send status	`FAILED`	Delivery failed; no bounce event
Voice note status	`VOICE_NOTE_UNPROCESSED`	Voice note received; awaiting human review

1.6.14 Work Order Item States

Value	Meaning
`PENDING`	Assigned; not started
`IN_PROGRESS`	Being worked on
`COMPLETED`	Finished; completion evidence recorded
`CANCELLED`	Cancelled with recorded reason

1.6.15 Group Billing Mode Values

Applicable to GROUP use type entries at S7 (§56):

Value	Meaning
`CONSOLIDATED`	One folio for the entire group

Value	Meaning
INDIVIDUAL	Each guest has their own folio
SPLIT	Some charges consolidated, others tracked individually

1.6.16 Amendment Path Identifiers

Value	Meaning
PATH_1	Simple folio addition — FOM authority
PATH_2	Commercial amendment (rate/inclusion) — FOM minimum; GM for rate during stay
PATH_3	Full renegotiation — GM for rate; FOM for inclusions

1.6.17 Post-Closure Access Levels

Level	Actor	Permitted
POST_CLOSURE_L1	Any authorised staff	Read-only access to sealed engagement
POST_CLOSURE_L2	FOM	Additive records only (disputes, credit notes, post-stay charges). All carry actual transaction date.
POST_CLOSURE_L3	GM	Corrections to original records — additive with own date, original preserved. No deletion. No backdating.

1.6.18 Commission-Due Record Status Values

Value	Meaning
PENDING	Commission-due record produced; awaiting settlement
SETTLED	Commission payment reconciled
RATE_MISSING	Agent profile has no commission rate configured; record produced as seam activation flag only

> *Commission-due records are produced only when an agent profile has a commission rate configured in the Admin Console. When no rate is configured — the current Legphel operating reality — no commission-due record is produced and no process is blocked by its absence (§50.5, MOM-007 T20).*

1.6.19 Canonical Record Type Names

The following are the canonical record type names as registered in §70 (Canon v2.4). All code, service methods, and audit events must use these exact names:

Inquiry · Entry · Segment · Availability Configuration · Quotation · Speculative Hold · Committed Hold · Reservation · Commitment Snapshot · Folio (Provisional) · Folio (Live) · Invoice · Payment Record · Credit Note · Commercial Adjustment Entry · Commission-Due Record · Credit Extension Ceiling Record · Credit Ceiling Threshold Event · Work Order · Work Order To-Do Item · Handoff Record · Communication Record · AI Draft Record · Human Decision Record · Correction Record · AI Audit Supplement Record · Staff Listening Summary Record · Dispute Record · Service Recovery Record · Resolution Bundle (v2.4) · DisputeGateOverrideRecord (v2.3) · Night Audit Record · Session Event Record (v2.5) · Processing Lock Record

Full mutation rules, lifecycle columns, and create/seal/archive rules for each record type are covered in Part 2 (Schema, §§2.1–2.20). The §71 mutation rules and §74 stage-to-record matrix are derived at the Part 2 gate.

---

§1.7 — Module Boundaries and Dependency Rules

1.7.1 Layer Hierarchy Within Modules

Within every operational module, the following layer hierarchy is mandatory. No layer may skip another layer:

```
Controllers (HTTP adapters)
  → Services (business orchestration)
    → Policies and Engines (rule evaluation and computation)
      → Models (Prisma data access)
```

Controllers are thin adapters. They parse requests, call services, and format responses. Business logic does not appear in controllers. Validation middleware executes before the controller handler. Auth middleware executes before the controller handler. Controllers do not contain conditionals that implement business rules.

Services orchestrate business operations. They call policies, engines, and models in the correct sequence. They do not contain raw SQL. All database access passes through Prisma models. Services emit trace events in the same transaction as state changes.

Policies evaluate rules and return outcomes. They may be called directly in tests without service orchestration. Calling a policy directly still triggers enforcement — policy enforcement is not a side-effect of service orchestration.

Engines compute deterministic outputs from defined inputs. Engines are pure functions over their inputs and configuration parameters. Engine behaviour is documented as hardcoded vs configurable per §1.4.4.

Models are Prisma data access objects. No business logic appears in model definitions beyond Prisma schema-level constraints.

1.7.2 Service Classification and Import Rules

Services are classified in three tiers. Import rules are enforced:

Service Tier	May Import	May Not Import
Infrastructure services	Nothing in the tier hierarchy	Domain services, Application services
Domain services	Infrastructure services	Application services
Application services	Domain services, Infrastructure services	—

No circular dependencies are permitted in any direction. A dependency graph that contains a cycle in any tier is an architectural defect.

1.7.3 Admin Console Namespace Separation

The Admin Console and operational code exist in separate organisational units (namespaces, module folders, or equivalent structural separation in the codebase). This separation is enforced, not just conventional.

Admin code must not import operational services. Any import of an operational service module within admin code is a boundary violation.

Operational code must not call admin services. Operational services read configuration values from configuration tables through standard Prisma access. They do not call admin service methods.

Configuration table reads are permitted from operational code. Operations may read from configuration tables. The direction of data flow for configuration is: Admin Console writes → configuration table → operational code reads. Operational code does not write to configuration tables.

1.7.4 Stage-Anchored Decision Rule

Every code path that creates, modifies, or transitions an operational record must have an active stage context (§7). Code that creates an operational record without a stage context is an architectural violation. The exception categories — post-closure access and night audit cross-stage operations — are governed and carry their own stage-context equivalent (entry-anchored post-closure access; stage-specific effects per stage of each active entry for night audit).

Admin configuration changes are the single category of action that genuinely sits outside the stage model. That separation is structural and intentional. Configuration changes affect how stages behave at runtime. They do not participate in stage workflow.

1.7.5 Engagement Use Type Conditional Logic

The engagement use type attribute on the Entry drives conditional logic at stages where behaviour diverges. This conditional logic is implemented within the standard stage processing services — not through separate parallel service trees or duplicate stage implementations. One stage, one state machine, one service path with conditional branches for use type variation where the Canon specifies variation.

---

§1.8 — Technology Stack and Runtime Environment

1.8.1 Backend Runtime

Component	Technology	Notes
Runtime	Node.js	JavaScript (`.js` files)
Framework	Express.js	HTTP server, routing, middleware
Language	JavaScript	Not TypeScript on the backend. Frontend is TypeScript.

1.8.2 Database and ORM

Component	Technology	Notes
Database	PostgreSQL	Primary data store. Single-tenant deployment. No `tenant_id` column.
ORM	**Prisma**	Schema-as-source-of-truth. `prisma migrate` is the migration toolchain. Prisma-generated typed client for all database access. No raw SQL in services.

ORM enforcement: All database access in services passes through Prisma models. No direct SQL queries in service or controller code. Schema migrations are generated by `prisma migrate` from the Prisma schema file. The Prisma schema file is the authoritative definition of the database structure.

1.8.3 Job Queue

Component	Technology	Notes
Durable job queue	**pg-boss**	PostgreSQL-native. No Redis dependency. Uses the existing PostgreSQL instance. Durable job scheduling and polling within a single-database operational profile. Future multi-property scale requires a formal architecture decision — job queue technology is revisited at that point.

pg-boss scope: The Timer Engine is implemented on pg-boss. All time-governed events — hold expiry, quotation validity expiry, parking follow-up, no-show contact window, SLA monitoring, feedback solicitation, processing lock expiry — register with pg-boss. No worker or engine manages its own scheduling independently of pg-boss (§19, Timer Engine universal scope).

1.8.4 Frontend (Boundary Reference Only)

Component	Technology	Notes
Frontend framework	Next.js / React	TypeScript. Excluded from DEV-SPEC scope. Referenced here for boundary clarity only.

The frontend is excluded from DEV-SPEC-001. All frontend implementation concerns are outside the scope of this specification. DEV-SPEC-001 defines the backend API contract, service behaviour, and data models. The frontend consumes these.

1.8.5 Authentication

Mechanism	Detail
Primary auth model	PIN-based fast user switching. Each staff member has an individual PIN. Switching terminals takes seconds. Session is paused on switch; resumes on PIN re-entry. Eliminates credential sharing motivation.
Session tokens	JWT for session state management.
Secrets	All secrets, connection strings, and environment-specific values in `.env`. No hardcoded secrets in code.

1.8.6 Communication Infrastructure

Channel	Infrastructure	Notes
Outbound email	Amazon SES	Domain authentication (SPF/DKIM/DMARC). Queue-based sending.
Inbound email	IMAP polling	Configurable polling interval. Includes dedicated OTA inbox. Webhook seam for future real-time upgrade.
WhatsApp	WhatsApp Business API via BSP	Bidirectional. Locked in MOM-007 (T08).

Channel	Infrastructure	Notes
AI agent LLM	External LLM API	Locked in MOM-007 (T19). ContextAssemblyService assembles engagement context per inbound message.

1.8.7 Abstracted Interfaces

The following capabilities are abstracted behind interface layers. Implementation details are covered in Part 11 (Integration Interfaces):

Interface	Covered in
File storage	§11.5
Document generation	§11.8

1.8.8 Deployment Model

Property	Value
Tenancy	Single-tenant. One property. One configuration namespace.
Multi-property	Not designed for and not designed against in this version. No `tenant_id` column. No multi-tenancy partitioning. A future multi-property extension is a formally scoped architectural decision (§17).

1.8.9 Existing Error Classes to Preserve

The following error classes exist in the codebase and must be retained in their current form, extended as specified in §1.5:

`AppError`, `ValidationError`, `NotFoundError`, `PolicyViolationError`, `AuthorizationError`, `StateTransitionError`

---

§1.9 — Engagement Use Type Variation Reference

1.9.1 Derivation Source

This section derives from §80 (Engagement Use Type Variation Matrix) as extended by Canon v2.4 (Group column — REV-B10-V24-01). The Group column derives from §56 (Group Booking Management).

This section is a reference summary. The governing operational doctrine for each use type lives in its respective stage charter variation blocks and cross-stage mechanism variation blocks. For Group, §56 is the authoritative section for full detail including authority chain, FOC policy, billing transition rules, bulk entry tool, and coordinator governance.

1.9.2 What Does Not Vary by Use Type

The following architectural behaviours are invariant across all five use types. They must not be conditioned on use type in implementation code:

State machine transitions · Policy enforcement · Audit capture · Ownership rules · Amendment traceability · Timer governance · Handoff structure · Escalation paths · Dispute framework · DEFICIENT flag doctrine · Credit ceiling

monitoring · OTA_SOURCE flag handling

1.9.3 What Varies: Full Matrix (Five Use Types × Nine Stages)

Stage	ACCOMMODATION	CONFERENCE	APARTMENT	CATERING	GROUP
S1	Room type selection; date range capture; DEFICIENT flag surfaced in availability results	Hall and seating configuration; attendee count capture; F&B inclusion selection from inclusion taxonomy; contract detection (existing contract routes S2 to confirmation step)	Duration capture; apartment unit selection; apartment preferences	Delivery location; F&B composition; logistics notes	Group inquiry created; expected room count and composition captured; multiple contacts recorded (agent as commercial counterparty, coordinator as operational contact, group leader if known); use type and corporate context selected
S2	Standard rate negotiation per pricing pipeline	Existing contract = confirmation step rather than negotiation; volume pricing bands; F&B composition options	Duration-based rate tier applies; lease terms negotiated; deposit terms set	F&B-focused pricing; no room component	Rate negotiated for the group; volume pricing bands may apply; FOC entitlement calculated and presented for GM approval; quotation reflects group terms
S3	Room committed hold; advance payment per policy; credit ceiling if credit extended	Coordinator formally confirmed with contacts and authority scope; milestone payment schedule; work order initiated; credit ceiling set for government clients	Security deposit collected and tracked separately from advance payment; billing cycle configured (weekly/monthly); credit ceiling covers full tenancy period	Kitchen capacity hold; no room hold required	Committed hold on full room block; advance payment per group policy; coordinator formally confirmed with authority scope; work order initiated if group includes conference or package services

Stage	ACCOMMODATION	CONFERENCE	APARTMENT	CATERING	GROUP
S4	Standard confirmation; OTA_CONFLICT typing applied automatically if OTA_SOURCE present and overbooking detected	FOM verification of hall, seating, F&B, and equipment before S4 proceeds; multi-booking detection required	Lease confirmation; inventory locked for full tenancy duration	Catering commitment confirmed; no room inventory locked	Full group confirmation covers room block; **two paths:** (a) Early rooming list path — individual entries created at S4 from available rooming list; (b) Late rooming list path — group container confirmed, individual entries populated when list arrives; both paths governed through same state machine
S5	Room readiness; DEFICIENT flag acknowledgement if flag present; no-show contact and detection	Sub-phase A: site visit recorded as a formal event; Sub-phase B: work order to-do list prepared; credit ceiling proximity monitored	Unit readiness; extended-stay housekeeping schedule configured; kitchen inventory if applicable	Kitchen readiness; delivery logistics confirmed; no room to prepare; event cancellation follows cancellation mechanism	Rooming list arrival triggers bulk entry creation via bulk entry tool (Excel/CSV upload, copy-paste, or rapid manual keyboard entry); corporate group hierarchy-aware allocation applies (CEO → suite, directors → deluxe, staff → standard); pre-arrival timers fire per individual entry; partial no-show handling applies if any individuals do not arrive
S6	Identity verification per §16.1; room assignment; folio activation; H2 and H3 handoffs; DEFICIENT flag carried in H2; VIP notification if applicable	Hall setup verified before client arrives; VIP notification if applicable; folio activated with pre-confirmed charges	Lease acknowledgement event recorded; deposit confirmed; extended H2 housekeeping schedule passed to housekeeping	Delivery dispatch event; no room occupancy	Identity verification per §16.1 at S6 for each individual entry; each individual entry follows standard S6 protocol; VIP notification fires if any group member carries VIP tier

Stage	ACCOMMODATION	CONFERENCE	APARTMENT	CATERING	GROUP
S7	Daily room charges posted by night audit; missing-charges anomaly detection; credit ceiling monitoring; DEFICIENT condition resolution tracking	F&B cover tracking by meal period; coordinator amendments mediated by hotel staff; space state management	Periodic billing cycle (weekly or monthly invoicing); extended housekeeping SLA; security deposit held throughout tenancy	Delivery tracking; consumption recorded against work order allocations	Individual entries operate independently during stay; billing modes active: CONSOLIDATED (one group folio), INDIVIDUAL (each guest folio), or SPLIT (specified charges consolidated, others individual); billing mode transitions permitted at any stage from S2 onward with FOM authority and recorded reason; folio split event created as additive record when individual switches from consolidated — consolidated folio is not edited
S8	Standard checkout; room inspection; damage charge posting from damage rate list; DEFICIENT inspection review by FOM if flag was in place	Preliminary bill review with coordinator before event end; complimentary room on folio reconciled	Lease settlement; security deposit reconciliation (return minus deductions); deduction events recorded	Equipment return recorded; no room release	Group checkout: sequential (guests depart over time — each individual processed in standard S8 flow) or crowd checkout (simultaneous departure — advance bill preparation by night audit the night before; priority sequencing; parallel processing enabled); individual settlement per guest's billing mode; coordinator reviews consolidated group bill separately from individual settlements

Stage	ACCOMMODATION	CONFERENCE	APARTMENT	CATERING	GROUP
S9	Standard closure; commission-due record produced at closure if agent commission rate configured	Government payment path (portal reconciliation); post-event follow-up; commission-due if agent-mediated	Deposit return tracked; lease closure event; commission-due if agent-mediated	Equipment return tracking confirmed; commission-due if agent-mediated	Group closure requires all individual entries to reach terminal state; outstanding balances per individual and per group governed independently; for agent-mediated groups with commission rate configured: one commission-due record per engagement covers group totals (total group folio value, applicable rate, calculated amount); if individual-level billing, individual commission-due records reference group container; if no commission rate configured, no commission-due record and group closure not blocked; group closure IS blocked if a produced commission-due record carries RATE_MISSING status that has not been resolved

1.9.4 Use Type as Entry Attribute

The use type is an attribute on the Entry record:

- Selected at S1 during initial entry creation.
- Carried unchanged throughout the lifecycle.
- Not modifiable after S1 without re-entry through the amendment mechanism.
- Drives conditional behaviour at stages where the Canon specifies variation.
- Does not create a separate state machine. All use types pass through the same S1–S9 state machine, the same guards, the same audit infrastructure.

1.9.5 Combined Engagements

A single Inquiry may contain multiple Entries, each with a different use type. A corporate event that includes accommodation for attendees, a conference hall booking, and a catering delivery for an off-site dinner creates one Inquiry with three Entries — one ACCOMMODATION, one CONFERENCE, one CATERING. Each Entry has its own independent lifecycle. Each progresses through S1–S9 independently. The Inquiry groups them commercially.

The system detects related Entries under the same Inquiry and may surface cross-Entry coordination opportunities (shared work order items, combined billing, coordinated arrival/departure). Cross-Entry coordination is surfaced as information; it does not merge the lifecycle management of independent Entries.

1.9.6 Cross-References for Full Variation Detail

- ACCOMMODATION: base behaviour; no separate variation section required
- CONFERENCE: full detail in §60 (Conference Management)
- APARTMENT: full detail in §27 (Engagement Use Types — Apartment subsection) and applicable stage charters
- CATERING: full detail in §27 (Engagement Use Types — Catering subsection) and applicable stage charters
- GROUP: full detail in §56 (Group Booking Management)

Part 1 Review Flags — Resolved

All three review flags from Gate 1 deliberation were resolved in the Gate 1 Review session (MOM-ARCH-2026-013).

Flag	Section	Resolution
RF-1	§1.2.1	**Accepted.** MOM series declared as 003–012. The spec reflects the actual decision record at time of writing.
RF-2	§1.6	**"Inquiry" locked** as the canonical implementation term. MOM-007 §5.1 rename is not active. Canon consistently uses "Inquiry" throughout all 11 blocks; Canon supersedes MOM where the MOM was not absorbed into the Canon. If rename is executed in future, it requires a Canon revision across all 11 blocks before the spec follows.
RF-3	§1.6.19	**Session Event Record confirmed** as a formal canonical record type. Added to Canon v2.5 (CANON-V2.5-CHANGESET.md, REV-B11-V25-01 and REV-B11-V25-02). Added to §1.6.19 with v2.5 marker. Part 2 schema gate carries the Prisma model obligation.

End of DEV-SPEC-001-Part1.md

Gate 1 — Document Identity

Prepared by: Claude (AI Architectural Partner)

Date: 07 April 2026

Status: DRAFT — nothing is locked until Architect confirms